



SECURITY ASSESSMENT

Provided by Accretion Labs Pte Ltd. for Matrixdock
January 21, 2026
A25MAT1



Matrixdock

AUDITORS

Role	Name
Lead Auditor	Robert Reith (robert@accretion.xyz)
Auditor	David Ratiney (david@accretion.xyz)

CLIENT

Matrixdock (<https://www.matrixdock.com/>) is building RWA infrastructure such as tokenized Gold on chain. They hired Accretion Labs to conduct a security assessment of their RWA implementation.

ENGAGEMENT TIMELINE



AUDITED CODE

Program 1

ProgramID: 3cGCHMomYzqoSVYB7EcL1zvDFFuNRzflTWcKpdrx5u

Repository: <https://github.com/Matrixdock-RWA/RWA-Contracts>

ASSESSMENT

The security assessment of Matrixdock's RWA Solana smart contracts revealed a well-architected system with no high or critical severity vulnerabilities. We identified 10 security findings: 4 medium, 4 low, and 2 informational.

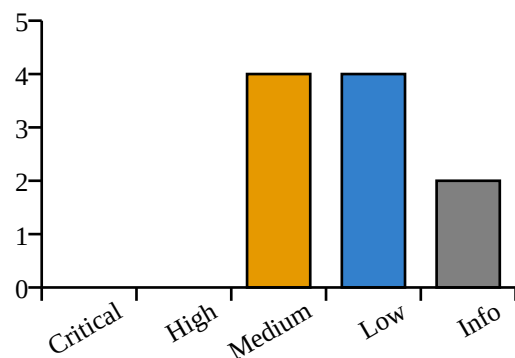
CODE ASSESSMENT

The code demonstrated solid Solana development practices with a clear separation of concerns between minting, redeeming, and pool management. The Anchor framework was used appropriately, though the codebase would benefit from additional on-chain validation checks and more defensive handling of account creation patterns.

KEY FINDINGS

Our top findings included a medium-severity rent stealing vulnerability in the request mint/redeem design, and other medium-severity issues around slippage enforcement not being validated on-chain, potential DoS through system program account creation, and temporary inconsistent state during two-step pool account updates. Most medium and low findings were acknowledged but accepted as known limitations.

SEVERITY DISTRIBUTION



ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Matrixdock RWA Contracts

Link: <https://github.com/Matrixdock-RWA/RWA-Contracts>

Commit: ca9d7f21

Program ID: 3cGCHMomYzqoSVYB7EcL1zvDFFuNRzflTWcKpdrx5u

Audit Result:

- **Audited Commit:** 16f79c285f782373f55ba1e6ecd9954365e27dc3
- **Build Hash:** not verified
- **Status:** not verified
- **Comment:** not verified

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-M1	System Program Create Account Usage Leads to Lamport Transfer DoS	MEDIUM	WONTFIX
ACC-M2	Request Mint/Redeem Design Potentially Enables Rent Stealing	MEDIUM	FIXED
ACC-M3	Slippage Not Enforced On Chain	MEDIUM	WONTFIX
ACC-M4	Two-Step Pool Account update Leads to Temporary Inconsistent State	MEDIUM	WONTFIX
ACC-L1	Lack of validation for target token (`for_token`)	LOW	WONTFIX
ACC-L2	Using program address as signer	LOW	WONTFIX
ACC-L3	Program Logs May Be Truncated	LOW	WONTFIX
ACC-L4	Lack of validation on `new_delay`	LOW	FIXED
ACC-I1	Single Step Owner Transfer	INFO	WONTFIX
ACC-I2	No Validation for Pool Accounts in Minter	INFO	WONTFIX

DETAILED ISSUES

ACC-M1 System Program Create Account Usage Leads to Lamport Transfer DoS

MEDIUM

WONTFIX

Description

We found that the program calls the System program's `create_account` instruction to initialize new accounts without checking the account's existing lamports. The System program's `create_account` instruction will fail and return an error when it tries to create an account which already contains any amount of lamports, which is a problem because anyone may transfer a small amount of lamports to the account to be created, effectively preventing the creation using `create_account`.

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-token/src/instructions/create.rs#L111-L123>

Relevant Code

```
/// create.rs L111-L123
create_account(
    CpiContext::new(
        ctx.accounts.system_program.to_account_info(),
        CreateAccount {
            from: ctx.accounts.payer.to_account_info(),
            to: ctx.accounts.mint_account.to_account_info(),
        },
    ),
    .with_signer(signer_seeds),
    lamports, // Lamports
    mint_size as u64, // Space
    &ctx.accounts.token_program.key(), // Owner Program
)?;
```

Mitigation Suggestion

To mitigate the issue, apply the manual transfer-allocate-assign pattern. First, transfer the required lamport amount to achieve rent exemption. This amount may be 0 if the account already has at least the amount of lamports required for the intended allocation size. Then, allocate the amount of bytes required and assign the account to the intended program.

Remediation

Marked as won't fix.

Description

We found that the Request for Mint/Redemption design allows users to potentially steal rent from the minter, even though the program tries to make the user initiate their ATA.

How the attack would work:

1. Request a Mint for a minimal amount (something like 1 lamport) 2. ATA gets created, but immediately close the ATA account 3. If the mint gets processed by some crank, and creates the ATA for the user if it doesn't exist, this means that the mint needs to pay for ATA creation 4. Once the ATA for the user is created by the mint crank, the user can close the account to reclaim the rent.

Same may work for redemptions as well.

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L345-L352>

Relevant Code

```
/// lib.rs L345-L352
#[account(
    init_if_needed,
    payer = requestor,
    associated_token::mint = for_token,
    associated_token::authority = requestor,
    associated_token::token_program = token_program_for_token,
)]
pub for_token_account: Box<InterfaceAccount<'info, TokenAccount>>,
```

Mitigation Suggestion

Either charge a fee (potentially refundable if accounts didn't need to be created) for minting and redemption, or make minting/redemption a single-step instruction called by the user.

Remediation

The issue has been fixed by requiring that the ATA account exists on Mint execution. Downgraded to medium as there is a minimum minting amount of \$10000.

ACC-M3 Slippage Not Enforced On Chain

MEDIUM

WONTFIX

Description

The minting program has a weird design, as slippage and preprice aren't validated on-chain, and no minting is actually done on a mint request or redeem request. So there are no real on chain protections for users to actually get the slippage they wanted, opening the possibility of a user getting worse execution than intended.

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L160-L168>

Relevant Code

```
/// lib.rs L102-L110
pub fn request_to_mint(
    ctx: Context<RequestMint>,
    transferred_token: Pubkey,
    for_token: Pubkey,
    amount: u64,
    preprice: u64,
    slippage: u64,
    timestamp: i64,
) -> Result<> {
/// lib.rs L160-L168
pub fn request_to_redeem(
    ctx: Context<RequestRedeem>,
    transferred_token: Pubkey,
    for_token: Pubkey,
    amount: u64,
    preprice: u64,
    slippage: u64,
    timestamp: i64,
) -> Result<> {
```

Mitigation Suggestion

Enforce slippage and execution details on chain.

Remediation

Marked as won't fix.

Description

We found that to update a pool account in the minter program, two steps are required:

1. set pool account a or b
2. set accepted by a or b

If those steps are called in two instructions, there is an in-between state where the pool account has been updated, but the accepted token list hasn't been updated yet.

This means that for this short time a pool may accept tokens that it wasn't intended for.

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L45-L99>

Relevant Code

```
/// lib.rs L45-L99
pub fn set_pool_account_a(ctx: Context<OnlyOwner>, new_pool_a: Pubkey) -> Result<()> {
    let state = &mut ctx.accounts.state;
    state.pool_account_a = new_pool_a;
    msg!("SetPoolAccountA: {}", new_pool_a);
    emit!(SetPoolAccountAEvent {
        pool_account_a: new_pool_a,
    });
    Ok(())
}

pub fn set_pool_account_b(ctx: Context<OnlyOwner>, new_pool_b: Pubkey) -> Result<()> {
    let state = &mut ctx.accounts.state;
    state.pool_account_b = new_pool_b;
    msg!("SetPoolAccountB: {}", new_pool_b);
    emit!(SetPoolAccountBEvent {
        pool_account_b: new_pool_b,
    });
    Ok(())
}

pub fn set_accepted_by_a(ctx: Context<OnlyOwner>, token: Pubkey, accepted: bool) -> Result<()> {
    let state = &mut ctx.accounts.state;
    if accepted {
        if !state.accepted_by_a.contains(&token) {
            require!(
                state.accepted_by_a.len() < MAX_ACCEPTED_TOKENS,
                ErrorCode::ExceedsMaxAcceptedTokens
            );
            state.accepted_by_a.push(token);
        }
    } else {
        state.accepted_by_a.retain(|x| x != &token);
    }
    msg!("SetAcceptedByA: {} -> {}", token, accepted);
    emit!(SetAcceptedByAEvent { token, accepted });
    Ok(())
}

pub fn set_accepted_by_b(ctx: Context<OnlyOwner>, token: Pubkey, accepted: bool) -> Result<()> {
    let state = &mut ctx.accounts.state;
    if accepted {
```

```

    if !state.accepted_by_b.contains(&token) {
        require!(
            state.accepted_by_b.len() < MAX_ACCEPTED_TOKENS,
            ErrorCode::ExceedsMaxAcceptedTokens
        );
        state.accepted_by_b.push(token);
    }
} else {
    state.accepted_by_b.retain(|x| x != &token);
}
msg!("SetAcceptedByB: {} -> {}", token, accepted);
emit!(SetAcceptedByBEvent { token, accepted });
Ok(())
}

```

Mitigation Suggestion

Instead of updating pools in a two-step instruction, use a single instruction that updates both pool account, and accepted by.

Remediation

This issue has been marked as won't fix.

Description

The `request_to_redeem` function accepts an arbitrary `for_token` mint and emits it in the `RedeemRequestEvent`.

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/ca9d7f21b97685d8fb7cb656d790964292a861f1/xaum-solan-a/programs/xaum-minter/src/lib.rs#L161>

Relevant Code

```
pub fn request_to_redeem(
    ctx: Context<RequestRedeem>,
    transferred_token: Pubkey,
    for_token: Pubkey,
    amount: u64,
    preprice: u64,
    slippage: u64,
    timestamp: i64,
) -> Result<()> {
    let state = &ctx.accounts.state;
    require!(
        state.accepted_by_b.contains(&transferred_token)
        && transferred_token == ctx.accounts.transferred_token.key(),
        ErrorCode::InvalidTokenForRedeeming
    );
    require!(
        state.pool_account_b == ctx.accounts.pool_account_b.key(),
        ErrorCode::InvalidPoolAccountB
    );
    let clock = Clock::get()?;
    require!(
        clock.unix_timestamp <= timestamp + DELAY_MAX,
        ErrorCode::InvalidTimestamp
    );
    let decimals = ctx.accounts.transferred_token.decimals;
    token_interface::transfer_checked(
        ctx.accounts.into_transfer_to_pool_b_ctx(),
        amount,
        decimals,
    )?;
    msg!(
        "RedeemRequest: transferred_token={}, for_token={}, requestor={}, pool={}, pool_ata={}, amount={}
, preprice={}, slippage={}",
        transferred_token,
        for_token,
        ctx.accounts.requestor.key(),
        state.pool_account_b,
        ctx.accounts.pool_token_account_b.key(),
        amount,
        preprice,
        slippage
    );
    emit!(RedeemRequestEvent {
        transferred_token,
        for_token,
        requestor: ctx.accounts.requestor.key(),
        pool: state.pool_account_b,
        pool_ata: ctx.accounts.pool_token_account_b.key(),
        amount,
        preprice,
        slippage,
```

```
});  
Ok()  
}
```

Mitigation Suggestion

Validate **for_token** against a whitelist of accepted tokens

Remediation

Marked as won't fix.

Description

We found that the program uses the program address itself (the address that the program will be deployed at) as a signer to verify the initialization instruction, and the create token instruction. However, this is bad style as the program key may be widely available to developers and may be used in testing.

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L295-L296> <https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-token/src/instructions/create.rs#L26-L27>

Relevant Code

```
/// lib.rs L295-L296
#[ account(address = crate::ID) ]
pub program: Signer<'info>,
/// create.rs L26-L27
#[ account(address = crate::ID) ]
pub program: Signer<'info>,
```

Mitigation Suggestion

It is recommended to use a separate admin key for authenticating admin-only functionality.

Remediation

Marked as won't fix.

Description

We found that the program uses the anchor `emit!` macro to log program events. However, solana logs may be truncated if the total log size in the same transaction exceeds 10kb. This means that logs may be only printed partially, or not at all, which can lead to issues in monitoring, debugging, indexing, and compliance.

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L48-L51> <https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L59-L61> <https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L78-L79> <https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L96-L97> <https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L147-L156> <https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L190-L210>

And also all of the calls to events in xaum-token (too many to list)

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-token/src/mtoken/events.rs#L1-L105>

Relevant Code

```
/// lib.rs L48-L51
msg!("SetPoolAccountA: {}", new_pool_a);
emit!(SetPoolAccountAEvent {
    pool_account_a: new_pool_a,
});
/// lib.rs L59-L61
emit!(SetPoolAccountBEvent {
    pool_account_b: new_pool_b,
});
/// lib.rs L78-L79
msg!("SetAcceptedByA: {} -> {}", token, accepted);
emit!(SetAcceptedByAEvent { token, accepted });
/// lib.rs L96-L97
msg!("SetAcceptedByB: {} -> {}", token, accepted);
emit!(SetAcceptedByBEvent { token, accepted });
/// lib.rs L147-L156
emit!(MintRequestEvent {
    transferred_token,
    for_token,
    requestor: ctx.accounts.requestor.key(),
    pool: state.pool_account_a,
    pool_ata: ctx.accounts.pool_token_account_a.key(),
    amount,
    preprice,
    slippage,
});
/// lib.rs L190-L210
msg!(
    "RedeemRequest: transferred_token={}, for_token={}, requestor={}, pool={}, pool_ata={}, amount={}",
    preprice={}, slippage={},
    transferred_token,
    for_token,
    ctx.accounts.requestor.key(),
    state.pool_account_b,
    ctx.accounts.pool_token_account_b.key(),
```

```

        amount,
        preprice,
        slippage
    );
    emit!(RedeemRequestEvent {
        transferred_token,
        for_token,
        requestor: ctx.accounts.requestor.key(),
        pool: state.pool_account_b,
        pool_ata: ctx.accounts.pool_token_account_b.key(),
        amount,
        preprice,
        slippage,
    });
}

/// events.rs L1-L105
use anchor_lang::prelude::*;

#[event]
pub struct SetOwnerRequest {
    pub old_owner: Pubkey,
    pub new_owner: Pubkey,
    pub et: i64,
}

#[event]
pub struct SetOwnerEffectuated {
    pub new_owner: Pubkey,
}

#[event]
pub struct SetOperatorRequest {
    pub old_operator: Pubkey,
    pub new_operator: Pubkey,
    pub et: i64,
}

#[event]
pub struct SetOperatorEffectuated {
    pub new_operator: Pubkey,
}

#[event]
pub struct SetRevokerRequest {
    pub old_revoker: Pubkey,
    pub new_revoker: Pubkey,
    pub et: i64,
}

#[event]
pub struct SetRevokerEffectuated {
    pub new_revoker: Pubkey,
}

#[event]
pub struct SetMessengerRequest {
    pub old_messenger: Pubkey,
    pub new_messenger: Pubkey,
    pub et: i64,
}

#[event]
pub struct SetMessengerEffectuated {
    pub new_messenger: Pubkey,
}

#[event]
pub struct SetDelayRequest {
    pub old_delay: i64,
    pub new_delay: i64,
    pub et: i64,
}

```

```

#[event]
pub struct SetDelayEffectuated {
    pub new_delay: i64,
}

#[event]
pub struct MintRequest {
    pub recipient: Pubkey,
    pub amount: u64,
    pub nonce: [u8; 32],
    pub et: i64,
}

#[event]
pub struct MintEffectuated {
    pub recipient: Pubkey,
    pub amount: u64,
    pub nonce: [u8; 32],
}

#[event]
pub struct Redeem {
    pub customer: Pubkey,
    pub amount: u64,
    pub data: Vec<u8>,
}

#[event]
pub struct ChangeMintBudget {
    pub delta: i64,
}

#[event]
pub struct BlockPlaced {
    pub user: Pubkey,
}

#[event]
pub struct BlockReleased {
    pub user: Pubkey,
}

#[event]
pub struct ForceTransfer {
    pub from: Pubkey,
    pub to: Pubkey,
    pub amount: u64,
}

```

Mitigation Suggestion

To mitigate the issue, use Anchor's `emit_cpi!` macro instead of the `emit!` macro for crucial event logs. This will ensure that logs will be passed as call data, which are not subject to truncation.

Remediation

Marked as won't fix.

Description

The `set_delay` instruction in the `xaum-token` program allows setting an arbitrary `i64` value for the new delay without any bounds checking. If `new_delay` is set to a negative value, the calculation for `next_et` (`current_unix_timestamp + state.delay`) can result in a timestamp that is already in the past. This effectively bypasses the intended timelock mechanism. An extremely large positive `new_delay` value (e.g., close to `i64::MAX`) can also be used by accident.

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/ca9d7f21b97685d8fb7cb656d790964292a861f1/xaum-solana/programs/xaum-token/src/instructions/authority.rs#L176>

Relevant Code

```
pub fn set_delay(ctx: Context<OwnerOp>, new_delay: i64) -> Result<()> {
    let state = &mut ctx.accounts.state;
    let clock = Clock::get()?;
    if state.next_delay_et == 0 {
        state.next_delay = new_delay;
        state.next_delay_et = clock.unix_timestamp + state.delay;
        emit!(SetDelayRequest {
            old_delay: state.delay,
            new_delay,
            et: state.next_delay_et,
        });
    } else {
        require!(
            state.next_delay_et <= clock.unix_timestamp,
            ErrorCode::NotEffective
        );
        require!(state.next_delay == new_delay, ErrorCode::RequestMismatch);
        state.delay = state.next_delay;
        state.next_delay_et = 0;
        emit!(SetDelayEffectuated { new_delay });
    }
    Ok(())
}
```

Mitigation Suggestion

Clamp the `new_delay` value e.g

```
require!(new_delay >= 0, ErrorCode::NegativeDelayNotAllowed);
require!(new_delay <= MAX_ACCEPTABLE_DELAY, ErrorCode::DelayExceedsMaximum); // Where `MAX_ACCEPTABLE_DELAY`
is a reasonable positive value
```

Remediation

Fixed in commit [f298471ef0d728d22ceca70ec96b32aa26ce5ef8](#).

Description

We found that the program uses a single step authority transfer design, which may be dangerous when the new owner contains a typo, copy paste error, or is accidentally set to the wrong account

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L39-L43>

Relevant Code

```
/// lib.rs L39-L43
pub fn transfer_ownership(ctx: Context<OnlyOwner>, new_owner: Pubkey) -> Result<()> {
    let state = &mut ctx.accounts.state;
    state.owner = new_owner;
    Ok(())
}
```

Mitigation Suggestion

It is better to implement a two step ownership transfer process, where the original owner retains ownership until the new owner explicitly calls the program to accept the ownership transfer.

Remediation

Marked as won't fix.

Description

We found that the provided pool accounts are never validated, neither in initialization, nor in the direct assignment instructions.

Location

<https://github.com/Matrixdock-RWA/RWA-Contracts/blob/94cb410310b8aaf24312f58a523af1cfd17116cc/xaum-solana/programs/xaum-minter/src/lib.rs#L14-L37>

Relevant Code

```
/// lib.rs L14-L37
pub fn initialize(
    ctx: Context<Initialize>,
    pool_account_a: Pubkey,
    pool_account_b: Pubkey,
    tokens_accepted_by_a: Vec<Pubkey>,
    tokens_accepted_by_b: Vec<Pubkey>,
) -> Result<()> {
    require!(
        tokens_accepted_by_a.len() <= MAX_ACCEPTED_TOKENS,
        ErrorCode::ExceedsMaxAcceptedTokens
    );
    require!(
        tokens_accepted_by_b.len() <= MAX_ACCEPTED_TOKENS,
        ErrorCode::ExceedsMaxAcceptedTokens
    );
    let state = &mut ctx.accounts.state;
    state.owner = *ctx.accounts.owner.key;
    state.pool_account_a = pool_account_a;
    state.pool_account_b = pool_account_b;
    state.accepted_by_a = tokens_accepted_by_a;
    state.accepted_by_b = tokens_accepted_by_b;
    state.bump = ctx.bumps.state;
    Ok(())
}
```

Mitigation Suggestion

Add some account validation that validates at least that the provided pool accounts belong to the expected program and have the correct discriminator.

Remediation

Marked as won't fix.

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:

- Solana-specific vulnerabilities
- Access control issues
- Arithmetic errors and precision loss
- Race conditions and MEV opportunities
- Logic errors and edge cases
- Performance optimization opportunities
- Invariant violations
- Account confusion vulnerabilities
- Authority check omissions
- Token22 implementation risks and SPL-related pitfalls
- Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.